

Chaos Theory Task

Student Name: Donia Mohamed Elrashidy

Date: 16 July 2026

Part A — Discrete Chaotic Systems

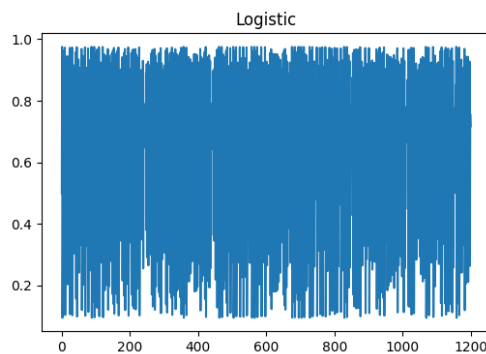
A.1 Logistic Map

The logistic map is defined as:

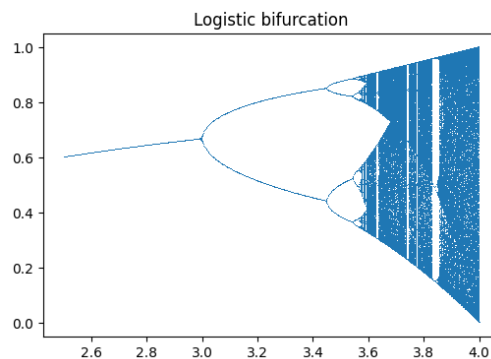
$$x_{n+1} = r \cdot x_n \cdot (1 - x_n)$$

- **State variable:** $x \in [0,1]$
- **Control parameter:** r (growth rate)
- **Initial condition:** $x_0=0.5$
- **Iterations:** 1200 total, discard first 300 as transient

Time Series for $r=3.9$:



Bifurcation Diagram for $r \in [2.5, 4.0]$:



At first, the system settles to a few values, but as r increases it becomes more complex and eventually appears chaotic (near $r \approx 3.5$). This shows that a small change in the parameter can greatly affect the system's long-term behavior.

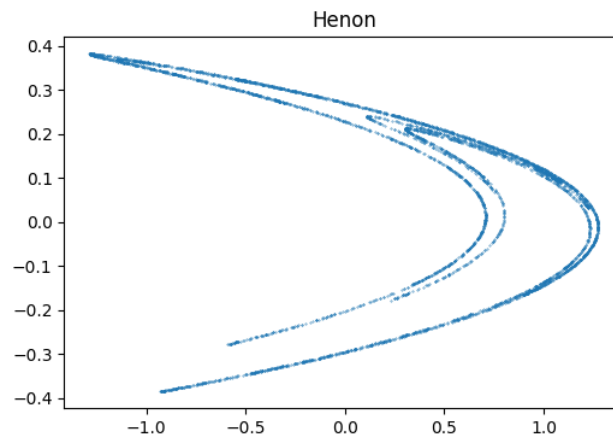
A.2 Hénon Map

A two-dimensional generalization of the logistic map:

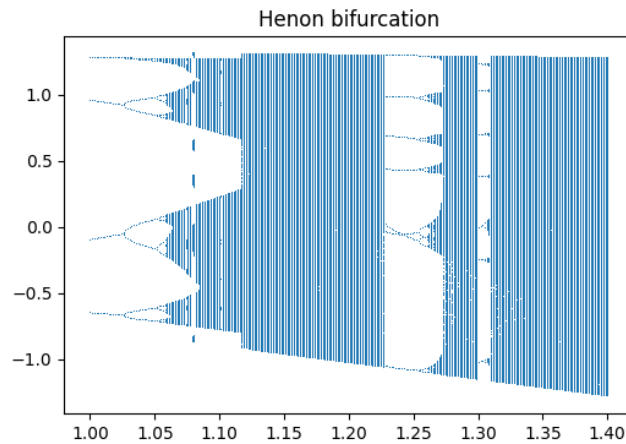
$$x_{n+1} = 1 - a \cdot x_n^2 + y_n$$

- **State variables:** (x,y)
- **Classic chaotic parameters:** $a=1.4, b=0.3$
- **Initial condition:** $(0,0)$
- **Iterations:** 5000, discard first 500

Henon Attractor:



Bifurcation Diagram for $a \in [1.0, 1.4]$ (b fixed at 0.3):



As the value of a changes, the behavior of the Hénon map also changes. Some regions show stable repeating values, while others become much more scattered, indicating chaotic behavior.

Part B — Continuous Chaotic Systems

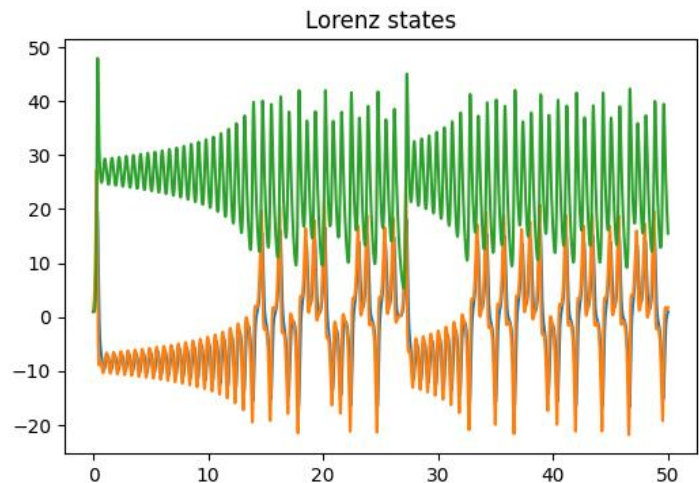
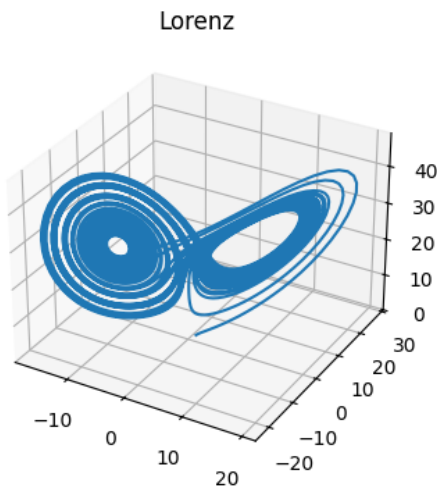
B.1 Lorenz System

Equations:

- $\dot{x} = \sigma(y - x)$
- $\dot{y} = x(\rho - z) - y$
- $\dot{z} = xy - \beta z$

- **Parameters:** $\sigma=10, \rho=28, \beta=8/3$
- **Initial condition:** $(1,1,1)$
- **Time step:** $dt=0.01$, **Simulation length:** $T=50$
- **Solver:** RK45 (solve_ivp)

3-D Phase Portrait and Time Series:



B.2 Rössler System

Equations:

$$\begin{aligned}\dot{x} &= -y - z \\ \dot{y} &= x + a \cdot y \\ \dot{z} &= b + z(x - c)\end{aligned}$$

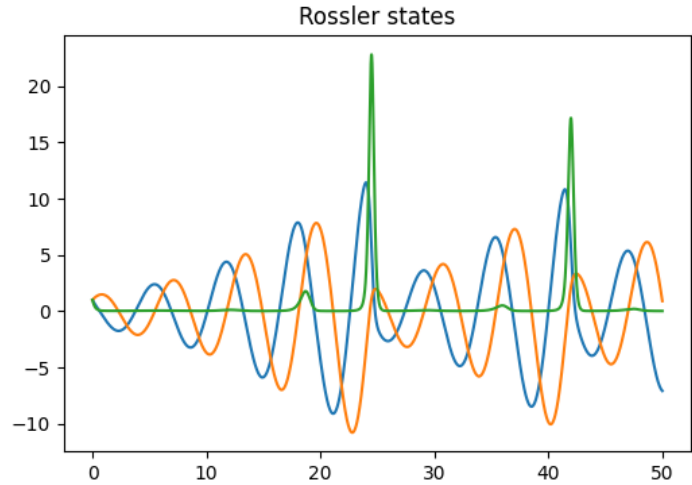
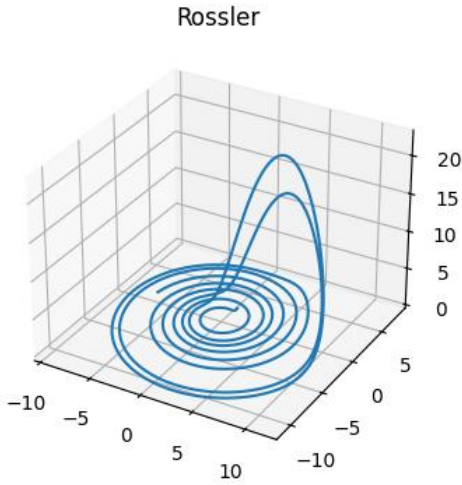
Parameters: $a = 0.2, b = 0.2, c = 5.7$

Initial condition: $(1,1,1)$

Time step: $dt = 0.01, T = 300$

Solver: RK45

3-D Phase Portrait and Time Series:



B.3 Chen System

Equations:

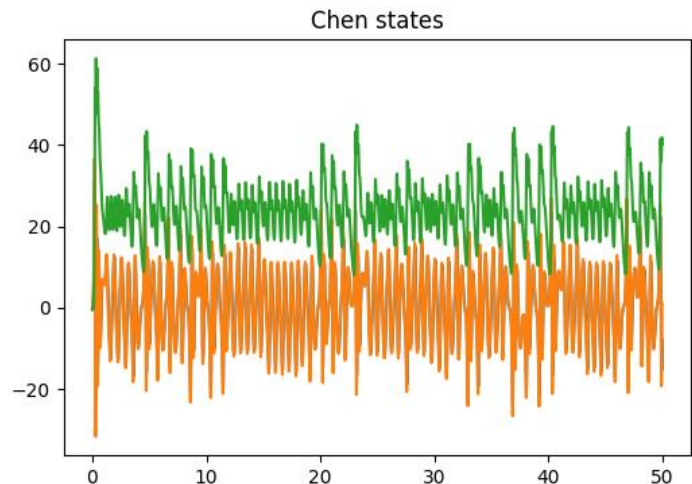
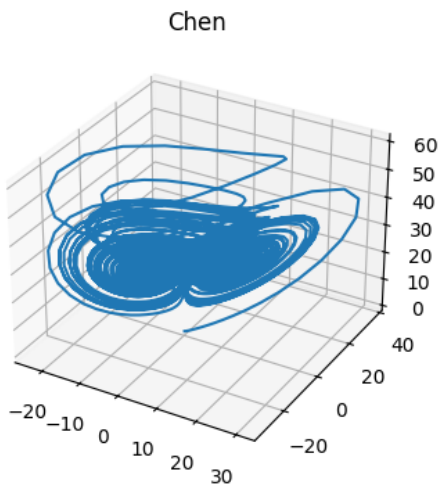
$$\dot{x} = a(y - x)$$

$$\dot{y} = (c - a)x - xz + cy$$

$$\dot{z} = xy - bz$$

- **Parameters:** $a=35, b=3, c=28$
- **Initial condition:** $(-0.1, 0.5, -0.6)$
- **Time step:** $dt=0.005$ (smaller due to stiffness), **T = 50**
- **Solver:** RK45

3-D Phase Portrait and Time Series:



B.4 Hyperchaotic Rössler System

Equations(4-D extension):

- $\dot{x} = -y - z$
- $\dot{y} = x + a \cdot y + w$
- $\dot{z} = b + xz$
- $\dot{w} = -c \cdot z + d \cdot w$

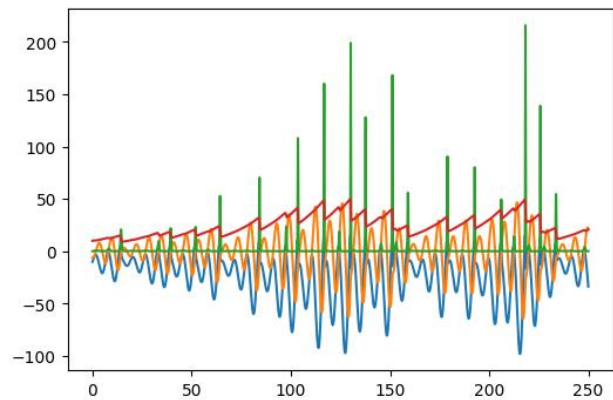
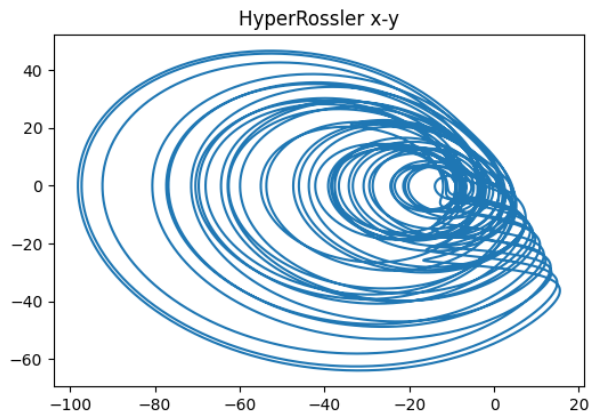
• Parameters: $a = 0.25, b = 3, c = 0.5, d = 0.05$

• Initial condition: $(-10, -6, 0, 10)$

• Time step: $dt = 0.01, T = 250$

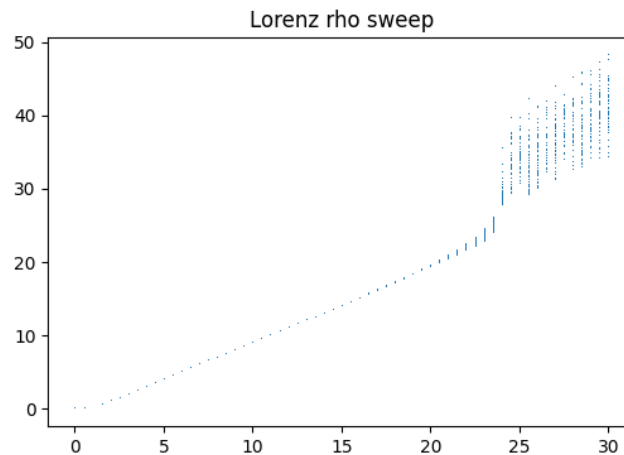
• Solver: RK45

2D Projections and Time Series:



Part C — Sensitivity Analysis

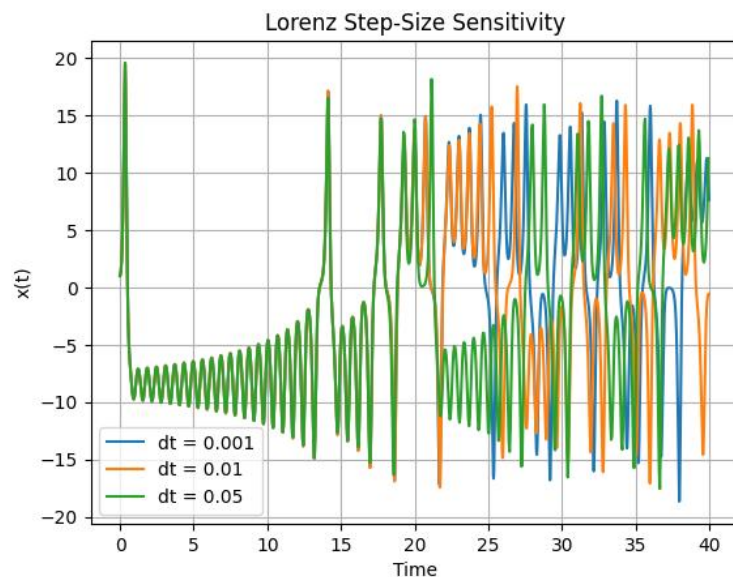
ρ -sweep plot for Lorenz plot:



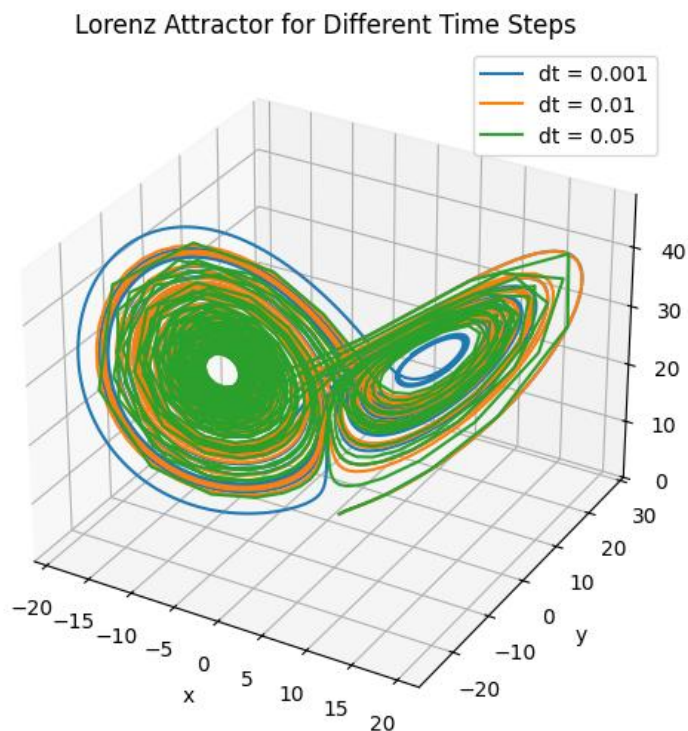
The ρ -sweep plot shows how the behavior of the Lorenz system changes as the parameter ρ increases. For smaller values of ρ , the system follows a simple and predictable pattern. As ρ becomes larger, the points spread out and multiple values appear, indicating more complex and chaotic behavior

The step-size comparison plots:

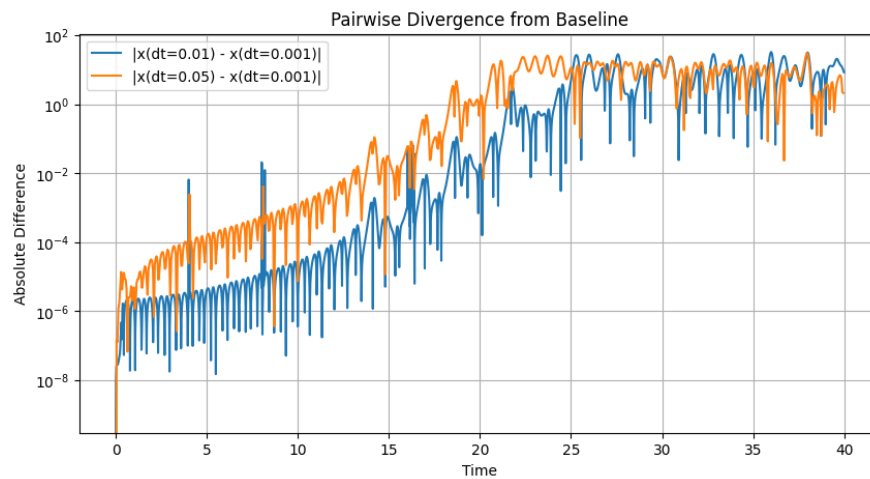
(i) overlay $x(t)$ for all three dt on one axis:



(ii) overlay the 3-D phase-space trajectories:



(iii) plot the pairwise divergence $|x_{dt}(t) - x_{baseline}(t)|$ vs. t :



At the beginning, all three simulations look almost the same because they start from the same initial conditions. After some time, the results begin to separate because different time step sizes introduce small calculation errors. Since the Lorenz system is chaotic, these small errors grow larger as the simulation continues. Among the tested values, $dt = 0.001$ gives the best accuracy, $dt = 0.01$ is still acceptable, while $dt = 0.05$ is too large to accurately represent the Lorenz attractor.

Appendix: Simulation Code

All simulation and plotting code is provided in the accompanying Jupyter notebook (TASK1_ChaosTheory.ipynb). The code uses standard Python libraries (numpy, matplotlib, scipy.integrate) and implements the required functions for each system.